

If we talk about Storage as a Service (STaaS) in cloud computing and if you are a cloud service provider, then a lot of your resources get wasted once fixed storage space is allotted to a client. Most of this space is rendered useless since the client doesn't use some of it and you are not allowed to use it either. To solve this or to minimise problems like this, it is preferable to use file data storage as a shared storage solution rather than object storage or a storage attached network (SAN).

In this article, I will try to lead you through the procedure of using and managing a file storage solution and also *qemu-img* virtualisation. I will even try to take you through some new techniques of disk virtualisation, which can be really useful at some point for your cloud server. I have used simple techniques, which can be easily understood and are not very striking but will really help you at different levels of storage system development.

All the tools used are generally available on most Linux systems; if not, these can be easily downloaded via your default online repository. The tested system for these commands is RHEL 7.0, but they will work on other Linux systems in the same way, with a little modification.

File storage: Solving the storage problem

If you are a STaaS provider, you will one day definitely come across the problem of a lot of your storage being wasted. This is because, if a user purchases 10GB of storage space from you, then you are forced to allocate him his 10GB, regardless of whether he will use all this space at once. Another challenge you will face frequently is that of maintaining the right amount of storage to provide the best service to your customers. As an example, if your data storage has 100GB of space left, someone may unexpectedly demand 200GB of storage from you, and you may have to refuse. But a good provider should fulfil all user requests and provide 24/7 service. So a good solution is to use file storage, with which you can scale your storage according to the data inside it and allocate space that you don't even have.

File storage relies on the fact that everything in your operating system is a file. In file storage, we create a file and use it as our virtual partition, then format it in the desired file system and mount it. All these operations are done in our real disk partition (*/dev/sda**) with some small tricks or manipulation. For this procedure, we use sparse images as our disk.

Sparse files: A sparse file is a specific type of file that aims to use the file system space more efficiently by using metadata to represent empty blocks. In sparse files, blocks are allocated and written dynamically as the actual data is written, rather than at the time the file is created. So, if you create a 10GB sparse file, it will

not even take 1MB of your disk space but in its property, it will show 10GB as that is the allocated space. So let's start the procedure.

A sparse file can be created by either using the *truncate* or the *dd* utility in Linux (other tools are also available).

```
$truncate --size=1GB test.img
```

The above command will create a 1GB sparse image *test.img*.

To get the same result with *dd*, use the following command:

```
$dd if=/dev/zero of=test.img bs=1024 count=0
seek=${1024*1000}
```

Here, *bs* is block size and the size is provided by *seek*.

You can also use the following code for simplicity:

```
$dd if=/dev/zero of=test.img bs=1 count=0 seek=1G
```

If you want to allocate the whole disk space at once (which is not a good solution in our case), you can use *fallocate*, as follows:

```
$fallocate -l 1G test.img
```



Note: Here's a little trick: Can I create a 2TB partition on my 1TB hard disk? The answer is, "Yes!" Since sparse files have the property of not taking up space during creation.

```
$truncate -size=2TB mylargefile.img
```

Create any desired file system on it, as follows:

```
$mkfs.ext2 mylargefile.img
$mount mylargefile.img /mnt/
```

You now have a 2TB partition.

After creating the file, you can format it and directly mount it in order to use it as your storage, but to enable it to do lots of disk operations it first has to connect to the loop device using the *losetup* utility. To attach the file to your loop device, use the following command:

```
$losetup -f test.img
```

...where *test.img* is your formatted file. To see all such files or block devices connected to the loop device, use the following command:

```
$losetup -a
```

You can easily grep the last created loop device by using the command below: